Software Ingeniaritza II

Proiektuaren diseinu patroiak

Market proiektua

Ikasleak: [Iago Vazquez eta Pablo Martineza]
Data: [13/11/2026]

Índice

1. Factory Method patroia **2**
1.1. Diseinua (UML) . . . 2
1.2. Inplementazioa . . . 2
1.2.1. Aurreko Kodea: . . . 2
1.2.2. Kode berria: . . . 2

2. Iterator patroia **4**
2.1. Diseinua (UML) . . . 4
2.2. Inplementazioa . . . 4
2.2.1. ExtendedIteratorSales: . . . 4
2.2.2. BLFacadeImplementation . . . 5
2.3. Exekuzioaren emaitza . . . 5

3. Adapter patroia **7**
3.1. Diseinua (UML) . . . 7
3.2. Inplementazioa . . . 7
3.2.1. Target . . . 7
3.2.2. Adapter . . . 8
3.2.3. Adaptee . . . 8
3.2.4. Kodearen azalpena . . . 8
3.3. Exekuzioaren emaitza . . . 10

1. Factory Method patroia

1.1. Diseinua (UML)

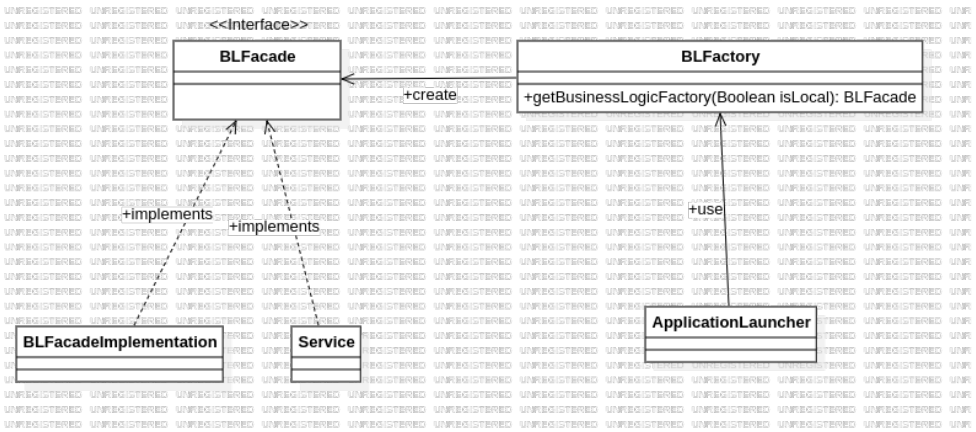


Figura 1: Factory Method UML diagrama

1.2. Inplementazioa

Factory Method patroia inplementatzeko, **BLFactory** klase berria sortu da (Creator rola), negozio logikaren instantziazioa **getBusinessLogicFactory(boolean isLocal)** metodoan zentralizatuz. Metodo honek, config.xml-eko isLocal balioaren arabera, BLFacadeImplementation lokala edo Web Service-aren proxy urrunekoa sortzen du, bi kasuetan **BLFacade** interfazea (Product rola) itzuliz.

Ondorioz, **ApplicationLauncher** klase nagusian, if/else logika ordezutu da.

1.2.1. Aurreko Kodea:

```
1  BLFacade appFacadeInterface;
2  if (c.isBusinessLogicLocal()) {
3      DataAccess da= new DataAccess();
4      appFacadeInterface=new BLFacadeImplementation(da);
5  }
6  else { //If remote
7      String serviceName= "http://"+c.getBusinessLogicNode() +":"+ c.
           getBusinessLogicPort()+"/ws/"+c.getBusinessLogicName()+"?
           wsdl";
8      URL url = new URL(serviceName);
9      QName qname = new QName("http://businessLogic/", "
           BLFacadeImplementationService");
10     Service service = Service.create(url, qname);
11     appFacadeInterface = service.getPort(BLFacade.class);
12 }
```

1.2.2. Kode berria:

```
1  boolean isLocal = c.isBusinessLogicLocal();
```

```
2 BLFactory factory = new BLFactory();
3 BLFacade appFacadeInterface = factory.getBusinessLogicFactory(
     isLocal);
```

Horrela, desakoplamendu osoa lortzen da, eta klase nagusia **DataAccess, URL, QName edo Service** ez ditu importatu behar. Gainera, etorkizunerako BLFacade elementu berriak gehitu nahi badugu soilik factory klasea ukitu behar dugu.

2. Iterator patroia

2.1. Diseinua (UML)

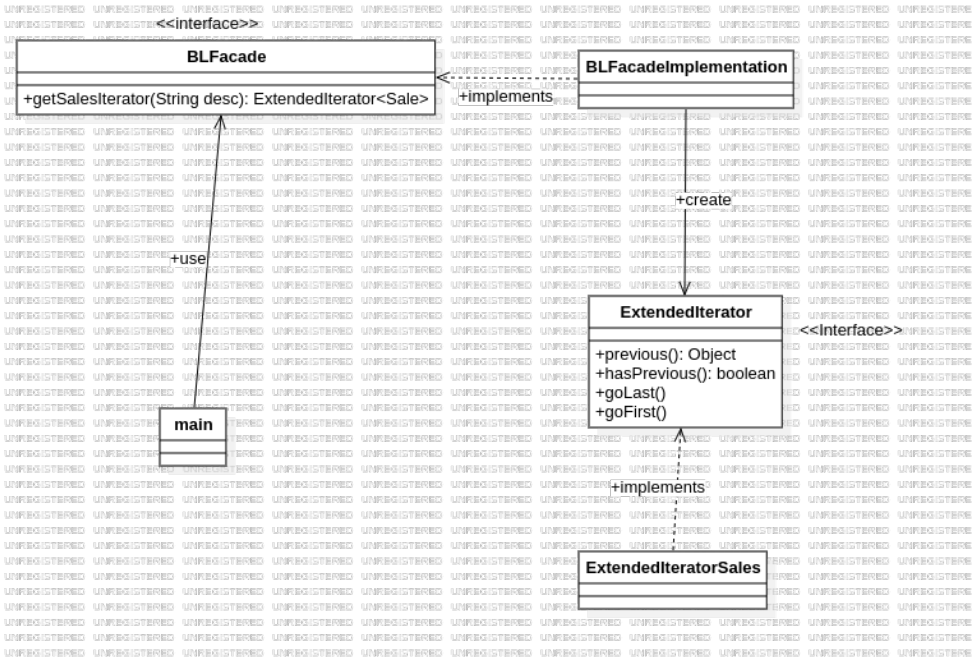


Figura 2: Iterator UML diagrama

2.2. Inplementazioa

Iterator Patroia implementatzeko, **ExtendedIterator<Object>** interfazea sortu da (Iterator interfazea hedatuz) eta ExtendedIteratorSales klasean inplementatu da, **Sale** objektuen bilduma bi norabideetan (aurrera zein atzera) eta hasierako/amaierako posizioetatik korritzeko aukera emanez

Negozio-logikan **getSalesIterator(String desc)** metodo berria gehitu da BLFacade interfazean eta BLFacadeImplementation klasean, datu-baseko salmentak lortu eta ExtendedIteratorSales instantzia bat itzultzeko. Horrela, bildumaren barne-egitura GUI-tik eta beste geruzetatik ezkutatzen da, elementuen gaineko nabigazio-eragiketak bildumaren inplementazio zehatzetik desakoplatuz.

2.2.1. ExtendedIteratorSales:

```
1     package businesslogic;
2
3 import java.util.List;
4 import domain.Sale;
5
6 public class ExtendedIteratorSales implements ExtendedIterator<Sale
     > {
7     private List<Sale> sales;
8     private int position = 0;
9
```

```
10      public ExtendedIteratorSales(List<Sale> sales) {
11          this.sales = sales;
12      }
13
14      @Override
15      public boolean hasNext() {
16          return position < sales.size();
17      }
18
19      @Override
20      public Sale next() {
21          Sale sale = sales.get(position);
22          position++;
23          return sale;
24      }
25
26      @Override
27      public boolean hasPrevious() {
28          return position >= 0 && position < sales.size();
29      }
30
31      @Override
32      public Sale previous() {
33          Sale sale = sales.get(position);
34          position--;
35          return sale;
36      }
37
38      @Override
39      public void goFirst() {
40          position = 0;
41      }
42
43      @Override
44      public void goLast() {
45          position = sales.size() - 1;
46      }
47  }
```

2.2.2. BLFacadeImplementation

```
1       @Override
2   public ExtendedIterator<Sale> getSalesIterator(String desc) {
3       dbManager.open();
4       List<Sale> sales = dbManager.getSales(desc);
5       dbManager.close();
6       return new ExtendedIteratorSales(sales);
7   }
```

2.3. Exekuzioaren emaitza

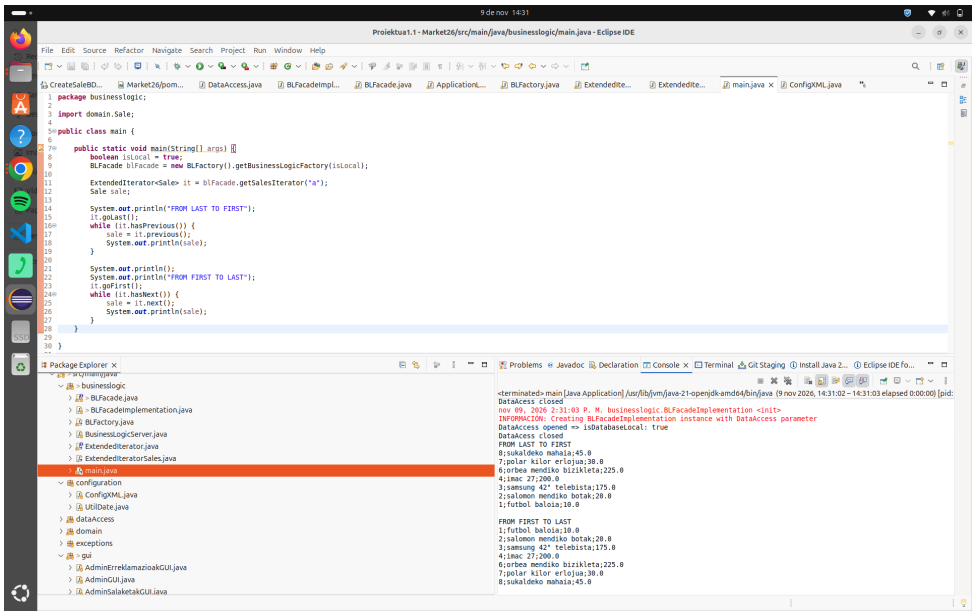


Figura 3: Iterator patroiaren emaitza

3. Adapter patroia

3.1. Diseinua (UML)

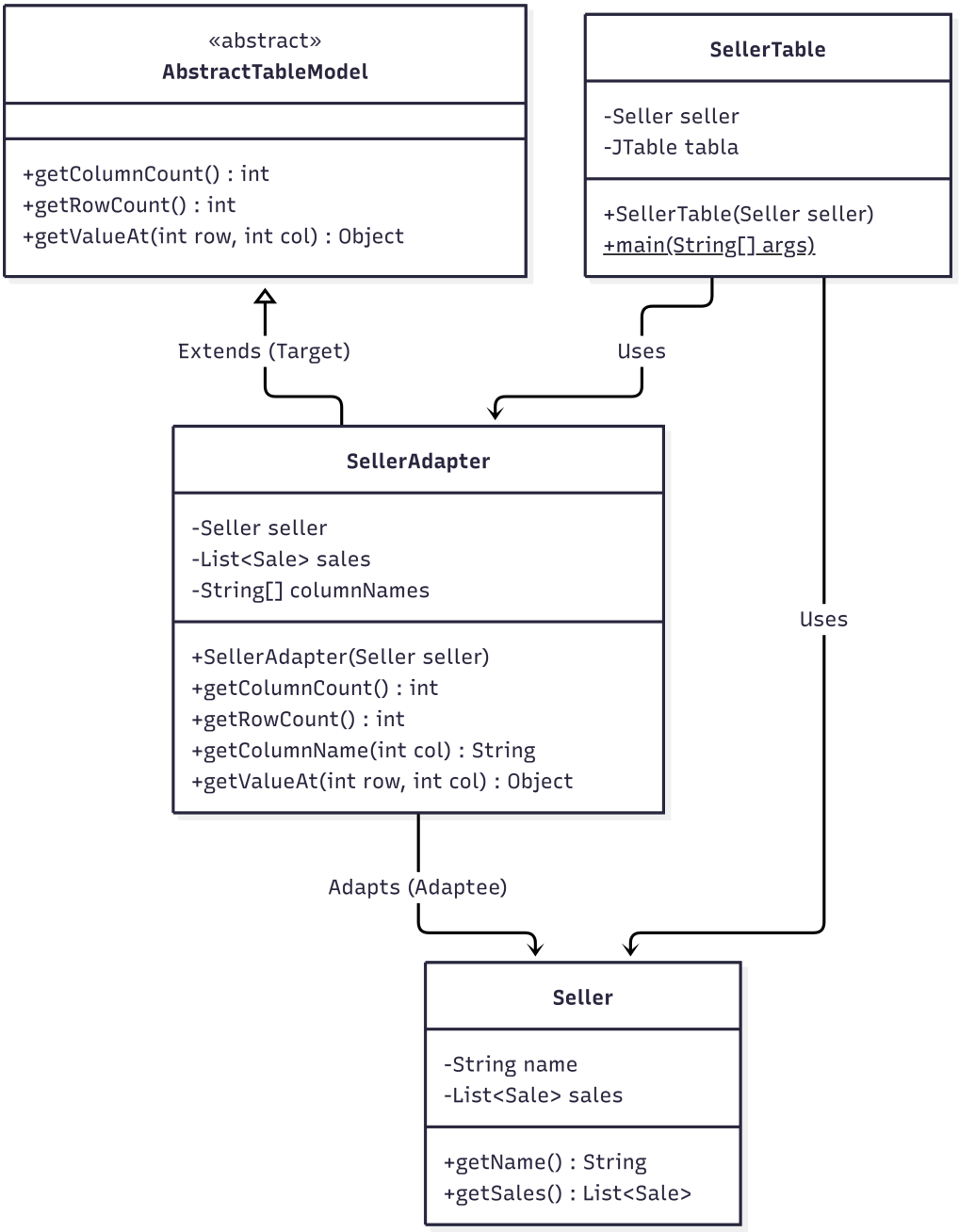


Figura 4: Adapter UML diagrama

3.2. Inplementazioa

3.2.1. Target

Azalpen orokorra: Programak datuak jasotzeko espero duen formatua edo txantiloia da. Ulertzeko erraza izateko: osagai grafikoak (`JTable` taulak) funtzionatu ahal izateko behar duen eredu estandarra da.

Inplementazioa: Kasu honetan, helburu estandar hori `AbstractTableModel` klasea da. Gure taula grafikoak klase hori behar du hiru gauza oinarrizko jakiteko: zenbat errenkada dauden, zenbat zutabe dauden eta gelaxka bakoitzean zer datu jarri behar den.

3.2.2. Adapter

Azalpen orokorra: *Adapter*-a zubi lanak egiten dituen klasea da. Bere helburua bezeroaren eta jatorrizko datuen arteko komunikazioa ahalbidetzea da. Funtsean, bezeroak egiten dituen eskaerak hartu eta gure datuek ulertzen duten formatura itzultzeaz arduratzen da.

Inplementazioa: Gure proiektuan, egokitzaile hori `SellerAdapter` klasea da. Klase honek `AbstractTableModel`-etik jasotzen du herentzia, eta bere barruan saltzailearen datuak gordetzen ditu. Taulak gelaxka baten datua eskatzen duenean, `SellerAdapter`-ak zerrendan bilatu eta dagokion atributua itzultzen dio taulari.

3.2.3. Adaptee

Azalpen orokorra: *Adaptee*-a erabili nahi ditugun datuak edo funtzionalitatea daukan jatorrizko klasea edo objektua da. Bere egitura ez da bezeroak espero duenarekin bateragarria, eta horregatik behar du tarteko egokitzaile bat datuak irakurri ahal izateko.

Inplementazioa: Gure proiektuan, egokitutako objektua `Seller` klasea da. `JTable` taulak ez daki zuzenean `Seller` objektu bat nola irakurri, horregatik, egokitzaileak objektu hau jasotzen du bere datuak banan-banan atera eta taulari emateko.

3.2.4. Kodearen azalpena

Gure inplementazioan, `SellerAdapter` klaseak `AbstractTableModel`-en nahitaezko hiru metodoak gainidazten ditu datuak taulan behar bezala erakusteko:

- `getColumnCount()`: 3 itzultzen du, hiru zutabe erakutsi nahi ditugulako.
- `getRowCount()`: Saltzailearen salmenta-zerrendaren tamaina itzultzen du.
- `getValueAt(int row, int col)`: Lehenik, errenkadaren arabera dagokion `Sale` objektua lortzen du. Ondoren, zutabearen arabera (0, 1 edo 2), objektu horren atributu zehatz bat itzultzen du.

Horrez gain, `SellerTable` klaseak Bezeroa bezala jokatzen du. Klase honetan `SellerAdapter` berria instantziatzen da eta zuzenean `JTable` osagaiari pasatzen zaio, honek datuak irakurri eta bistaratu ditzan.

```
1  public class SellerAdapter extends AbstractTableModel {
2      private Seller seller;
3      private List<Sale> sales;
4      private String[] columnNames = {"title", "description", "price"
           };
5
6      public SellerAdapter(Seller seller) {
7          this.seller = seller;
8          this.sales = seller.getSales();
9      }
10
11     @Override
```

```
12      public int getColumnCount() {
13          return 3;
14      }
15
16      @Override
17      public int getRowCount() {
18          return sales.size();
19      }
20
21      @Override
22      public String getColumnName(int col) {
23          return columnNames[col];
24      }
25
26      @Override
27      public Object getValueAt(int row, int col) {
28          Sale sale = sales.get(row);
29          switch(col) {
30              case 0: return sale.getTitle();
31              case 1: return sale.getDescription();
32              case 2: return sale.getPrice();
33              default: return null;
34          }
35      }
36  }
```

```
1   public class SellerTable extends JFrame {
2       private Seller seller;
3       private JTable tabla;
4
5       public SellerTable(Seller seller) {
6           super(seller.getName() + "'s sales ");
7           this.setBounds(100, 100, 700, 200);
8           this.seller = seller;
9
10          SellerAdapter adapt = new SellerAdapter(seller);
11          tabla = new JTable(adapt);
12          tabla.setPreferredScrollableViewportSize(new Dimension(500,
                70));
13
14          JScrollPane scrollPane = new JScrollPane(tabla);
15          getContentPane().add(scrollPane, BorderLayout.CENTER);
16      }
17
18      public static void main(String[] args) {
19          boolean isLocal = true;
20          businesslogic.BLFacade blFacade = new businesslogic.
               BLFactory().getBusinessLogicFactory(isLocal);
21          domain.Seller s = blFacade.isLogged("Aitor Fernandez", "
               aurrera");
22          SellerTable st = new SellerTable(s);
```

```
23          st.setVisible(true);
24      }
25  }
```

3.3. Exekuzioaren emaitza

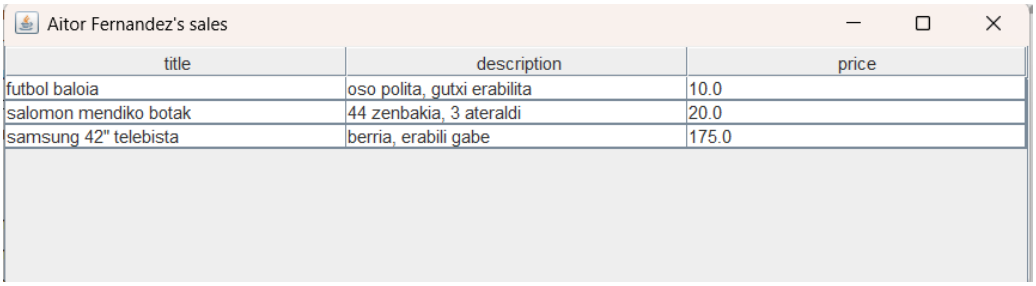


Figura 5: Adapter exekuzioaren emaitza